

AutoGraderPro Documentation

Welcome to the **AutoGraderPro** sub-project. Unlike the monolithic Streamlit application in the root directory, AutoGraderPro is designed as a decoupled, modern architecture utilizing a separate REST API backend, an autonomous command-line batch processor, and a standalone HTML/Vue.js frontend application.

This document serves as both the **Working Guide** and the **Code Explanation** strictly for the internal mechanisms of the AutoGraderPro ecosystem.

Architecture Overview

The AutoGraderPro ecosystem breaks the OMR grading process into distinct, scalable micro-services:

1. **api_server.py (The Backend Core)**: A Flask-based RESTful API that handles HTTP requests from the frontend, manages the local database (JSON files), and proxies image processing requests.
 2. **frontend_app.html (The Presentation Layer)**: A completely standalone, single-page application built using Vue.js for reactivity and Tailwind CSS for styling. It allows users to define templates, extract keys, and monitor grading through browser API calls.
 3. **batch_process.py (The Autonomous Pipeline)**: A headless command-line script designed to ingest hundreds of photographs from an input directory, grade them automatically based on the active template and master key, and export the annotated results to a destination folder.
 4. **src/image_core.py and src/region_scanner.py (The Mathematical Core)**: The isolated Computer Vision logic containing OpenCV mathematical operations that align and parse bubbles.
-

Environment Setup

Before running AutoGraderPro, you must configure a Python Virtual Environment and install the required dependencies.

1. Create a Virtual Environment

Navigate to the `AutoGraderPro` directory in your terminal and run the following command based on your Operating System:

- **Windows:**

```
python -m venv venv
```

- **macOS / Linux:**

```
python3 -m venv venv
```

2. Activate the Virtual Environment

You must activate the environment every time you work on this project.

- **Windows (Command Prompt):**

```
venv\Scripts\activate
```

- **Windows (PowerShell):**

```
venv\Scripts\Activate.ps1
```

- **macOS / Linux:**

```
source venv/bin/activate
```

3. Install Dependencies

Once the environment is activated, install all required packages via `pip`:

```
pip install -r requirements.txt
```

How to Run AutoGraderPro

1. Starting the API Server

The backend must be running for the frontend to save templates or process web-based grading:

```
python api_server.py
```

This initializes the Flask server (typically on `http://127.0.0.1:5000`) and handles Cross-Origin Resource Sharing (CORS) to allow the standalone HTML file to communicate with it.

2. Launching the Frontend

Because the frontend is a standalone HTML file utilizing native browser fetch requests, you do not need a web server to run it. Simply double-click `frontend_app.html` to open it in Chrome, Firefox, or Edge.

3. Running the Batch Processor (Headless Mode)

If you have 100 student exams to grade, doing it one by one in the UI is highly inefficient.

1. Ensure you have activated a template and a master key using the frontend or `app.py`.
2. Place all un-graded student exam photographs (JPG, PNG) into the `AutoGraderPro/input_docs/` folder.
3. Run the batch processor:

```
python batch_process.py
```

4. The system will iterate through every image. It will output annotated, graded images and an exported CSV data report into the `AutoGraderPro/processed_exports/` folder automatically.
-

Step-by-Step Usage Guide (Frontend)

The Vue.js frontend allows you to interactively build the spatial templates and answer keys that the system needs to grade exams.

1. Build a Blueprint (Template Setup)

Before anything else, you must define the physical layout of your OMR sheets.

1. Open `frontend_app.html` in your web browser.
2. Select the **Template Setup** tab.
3. Upload a completely blank examination sheet. The system will auto-align it on your screen.
4. **Draw Regions:** For every block of questions (or Roll Number), fill out the *Row Count* and *Options per Item*. Click and drag your mouse directly on the image to draw a blue bounding box strictly around the bubbles. Notice the box turns green when you release.
5. Click **Add to Blueprint**. Repeat this for every section on the page.
6. Type a name (e.g., “Final Exam 2026”) and click **Commit Blueprint**. Check the “Set as default Active Template” box so the batch processor knows to use it.

2. Extract the Master Answer Key

Once the template geometry is committed, you need to provide the “Truth” answers.

1. Select the **Extract Answer Key** tab.
2. Ensure your new blueprint is selected in the dropdown.
3. Enter an Examination Title.
4. Upload an OMR sheet that a teacher has physically bubbled in with 100% correct answers.
5. Click **Autogenerate Matrix**. The CV engine will scan the sheet and immediately show you exactly what it read in the data table below.
6. Click **Commit Evaluation Matrix** to save this truth data to `master_answers.json`. It is now ready for production grading.

3. Grading Individual Students (Web Processing)

You can instantly grade one-off student submissions directly in the browser.

1. Select the **Web Processing** tab.
 2. Ensure the correct “Target Evaluation Key” is selected.
 3. Upload a student’s completed OMR sheet.
 4. Click **Execute Grading Flow**. The system will grade the image and return a heavily annotated graphic showing the student’s score, green ticks for correct answers, red crosses for wrong ones, and blue circles highlighting what they *should* have bubbled in.
 5. Review the detailed data table at the bottom of the page to see exactly what they scored on every individual question.
-

Code Explanation & Inner Workings

This section breaks down the specific Python logic of the core files.

1. `api_server.py` (The State Manager)

This file replaces Streamlit’s internal state management with persistent JSON databases and standard REST architecture.

- **Data Persistence:** The API utilizes three local JSON files to act as a database:
 - `configs.json` (Stores Template Blueprints)
 - `master_answers.json` (Stores Generated Answer Keys)
 - `graded_results.json` (Stores Historical Grading Output)

- **The Blueprint Endpoints (`/api/templates`):**

When the frontend sends a POST request with newly drawn bounding box coordinates, `save_template()` loads `configs.json`, finds the active blueprint wrapper (e.g., “Midterm”), and injects the JSON array of coordinates (`zones`) directly into the file. It forcefully sets the `"is_default"` flag on the new template if requested, ensuring the batch processor knows which template is active.

- **The Grading Endpoint (`/api/grade`):**

When an image is posted to this endpoint:

1. It decodes the raw HTTP byte stream back into an OpenCV matrix using `cv2.imdecode`.
2. It immediately fetches the active blueprint and the active master key from the local JSON storage.
3. It triggers `image_core.locate_anchor_points` and constructs the warping m

atrix.

4. It pushes the matrix, the template zones, and the answers to `region_scanner.evaluate_template_zones`.
5. It returns a cleanly formatted JSON response containing the final score and a base64-encoded string of the heavily annotated image for the frontend to render natively in an `<img>` tag.

2. `batch_process.py` (The Data Pipeline)

This file completely bypasses the Flask API and the frontend, connecting directly to the CV logic for maximum processing speed.

- **Initialization Guards (`load_configs()`, `load_matrices()`):**

Before it attempts to process images, it aggressively checks if `configs.json` and `master_answers.json` exist, and if an active template/key is marked as `True`. If it fails this check, it fires `prompt_error_visual()`—a massive, custom OpenCV warning window drawn using NumPy zeros (`diag = np.zeros((300, 800, 3))`) to alert the user that they must configure the system first.

- **The Ingestion Loop:**

It iterates over `os.listdir('input_docs')`. For each valid image, it fires `execute_pipeline()`.

- **The Evaluation Execution:**

This function performs the standard alignment (`locate_anchor_points`, `calculate_warp_matrix`) and parses the sheet (`evaluate_template_zones`).

- **Destructive Rendering & Export:**

- It uses `cv2.putText` and `cv2.rectangle` heavily to permanently burn the final score, the roll number, and the diagnostic lines into the image pixel array.
- It uses `cv2.imwrite()` to dump this permanently altered image into `processed_exports/`.
- It immediately generates a detailed CSV manifest (`report_filename.csv`) using Python's native `csv` library, iterating over the `data_dump` dictionary returned by the scanner, providing a row-by-row breakdown of expected answers vs. scanned answers.

3. `frontend_app.html` (The Reactive UI)

Since AutoGraderPro does not use Streamlit, the user interface relies on Vue.js to manage the DOM instantly.

- **State Hooking:** Vue's `data()` method tracks things like the current tab, the `activeTemplate`, the arrays of `zones`, and the `finalScore`.
- **Interactive Canvas:** When defining a template, the uploaded image is drawn onto an HTML5 `<canvas>`. JavaScript mouse event listeners (`mousedown`, `mousemove`, `mouseup`) calculate the exact X/Y coordinate ratio based on the physical size of the canvas element versus the intrinsic size of the image, building the bounding box data to be POSTed to the Flask API.
- **Reactivity:** When the Flask API responds to a grading request, it updates the `this.evaluationResult` object. Vue automatically detects this change and instantly populates the dynamic data table showing the student's breakdown without requiring a page refresh.

4. Mathematical Engine (`src/` directory)

The `src/image_core.py` and `src/region_scanner.py` files encapsulate the same robust Computer Vision logic found in the root project (adaptive contrast boosting, geometric distance matching for corner detection, and virtual coordinate warping via `cv2.perspectiveTransform`).

By isolating these functions inside the `src` module folder, AutoGraderPro allows `api_server.py` and `batch_process.py` to utilize identical mathematical grading constraints without duplicating code or relying on the legacy UI state files.